Markov Chains

Where We Left Off

- Last week: pair each day with the next day, then compute the **conditional PMF** of tomorrow given today
- We decided what the categories were for the market example: up & down/flat, or 5 or 21 groups
- Given our current state, we had the probability of "transitioning" to another state: P(tomorrow | today)
- Market direction, 26,295 pairs of trading days:

Today \ Next	Down / flat	Up
Down / flat	0.496	0.504
Up	0.417	0.583

Each row sums to 1

Today: Markov Chains

- The **state space**: decide what counts as a state— what categories/bins/clusters will we include? (Later: order)
- The conditional PMF table is an estimate of a **transition matrix**
  - (Later: It counts as one if two assumptions hold: **Markov property** and **Time-homogeneity)**

https://smgroves.github.io/assets/html/ds5030_2026/activities/activity-markov-chain-walk/index.html

Every transition matrix is a conditional PMF; not every conditional PMF is a transition matrix

1. Sequences and the State Space

Sequences and Stochastic Processes

- A **sequence** is a collection of values ordered by a "time" index
  - Market returns, indexed by time
  - Taxicab location: where the taxis move over time
  - Music: chord progressions and melodies are sequences of notes
  - Text: a sequence of word tokens
- A **stochastic process** is a sequence X_1, X_2, ..., X_T whose values are random: given what we know, we cannot perfectly predict what comes next
- Markov chains are the workhorse model. Markov invented them to study how letters follow one another in text
- They foreshadow tools from machine learning: recurrent networks, hidden Markov models, attention, transformers

Today & Thursday: state space → sequence → transition counts → conditional probabilities → simulations → forecasts

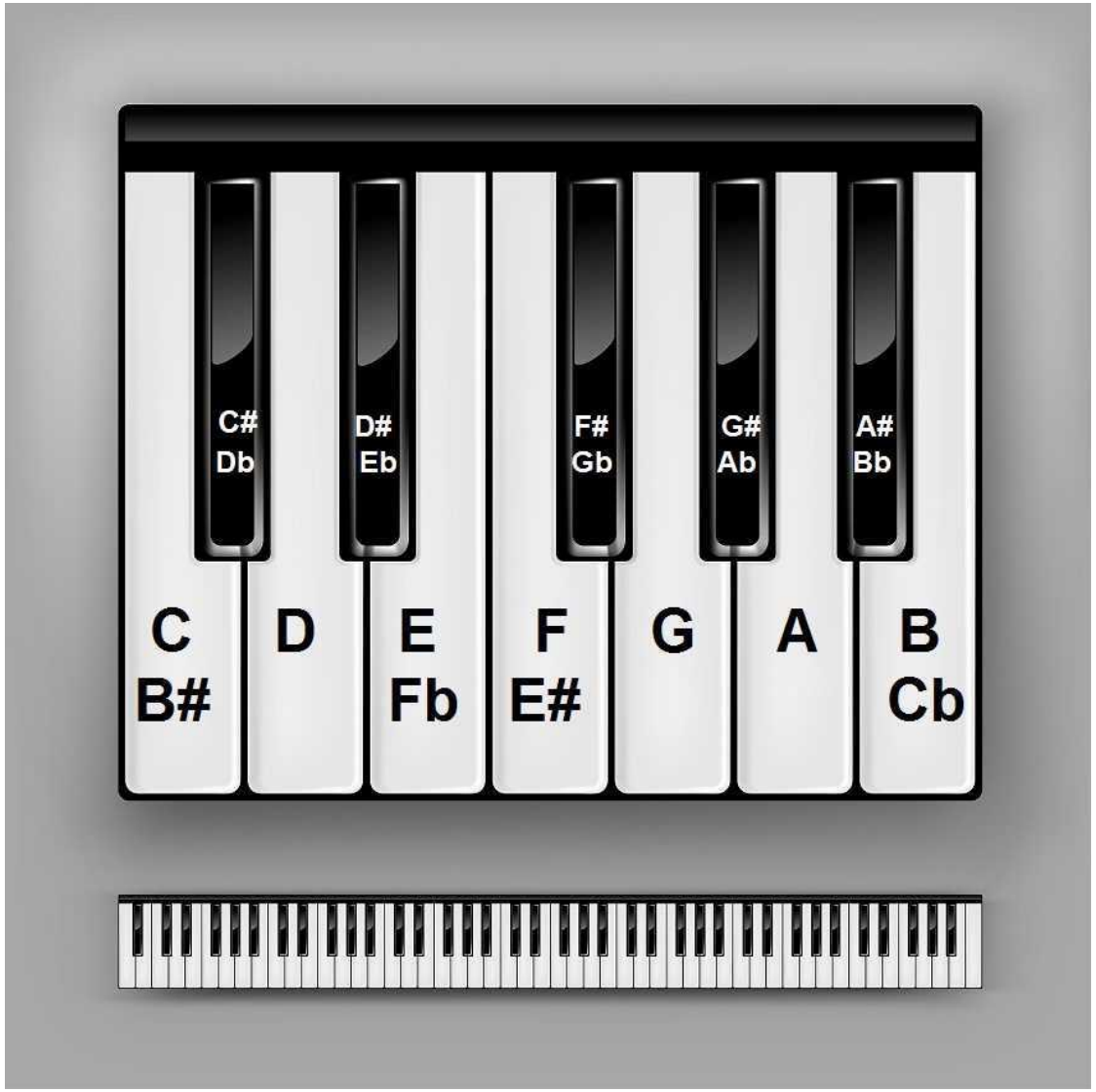
The State Space

- The **state space** is the set of values the sequence can take
  - Market returns: up & down/flat
  - Precision health: a patient's recent vital-sign measurements
  - Music: chords in a key
  - Text: a chunk of words ("To be or not ...")
- It should describe every relevant configuration of the system
- **Choosing the state space is a modeling decision**, and often the hard part. Much of what made large language models possible is better ways to define and model the states of text

Today & Thursday: state space → sequence → transition counts → conditional probabilities → simulations → forecasts

Example: Bach Chorales

- Music is "sound in time." Bach is close to a distillation of Western harmony
- Western music has 12 notes: C, C♯/D♭, D, D♯/E♭, E, F, F♯/G♭, G, G♯/A♭, A, A♯/B♭, B
- A **chord** is any set of notes played at the same time. Most combinations sound dissonant
- Chorale chords typically have three or four notes spaced by thirds: C major = CEG, A minor = ACE, G7 = GBDF
- The chords are the tokens we build the state space from



Bach: musical genius or artificial and confusing?

- Both sides: Bach can be “sublime” and can be detached, mathematical: https://lareviewofbooks.org/article/bach-psychology-gothic-sublime-just-human/
- Networks of note-to-note transitions (Bach as a chord generation machine): https://www.scientificamerican.com/article/secret-mathematical-patterns-revealed-in-bachs-music/
- “Bach's mathematical precision has a beautiful counterpoint in the emotional intensity of his music.” https://wnyc.org/story/john-eliot-gardiner-bach/
- Bach's music was artificial and confusing in style, and the notation of such elaborate ornaments (rather than leaving ornamentation to the performer, as was customary) obscured the melody and harmony. -- Johann Adolf Scheibe, Bach contemporary
- “That famous cello prelude, deconstructed” Vox: https://www.youtube.com/watch?v=UIge2mYdTtM
- AI generated music and Bach: https://www.ibiblio.org/gandhi/gandhi/hofstadter.htm ; https://computerhistory.org/blog/algorithmic-music-david-cope-and-emi/

From Rows to Sequences

```
bach = pd.read_csv('bach.data', header=None,
                   names=cols)
bach = bach.loc[:, ['song', 'time', 'chord']]

seqs = []
for song in bach['song'].unique():
    lt = list(bach.loc[bach['song'] == song,
                       'chord'])
    lt = [re.sub(r' ', '', ch) for ch in lt]
    lt = [re.sub(r'_', '', ch) for ch in lt]
    lt = [re.sub(r'M', '', ch) for ch in lt]
    seqs.append(lt)
```

- One list of chords per chorale
- The regexes only tidy the labels: F_M becomes F
- **Keep the songs separate.** The last chord of one chorale is not followed by the first chord of the next

Chorale 6 begins:

```
Gm Gm Gm Gm7 Cm Cm7 D D7 Gm D7 ...
```

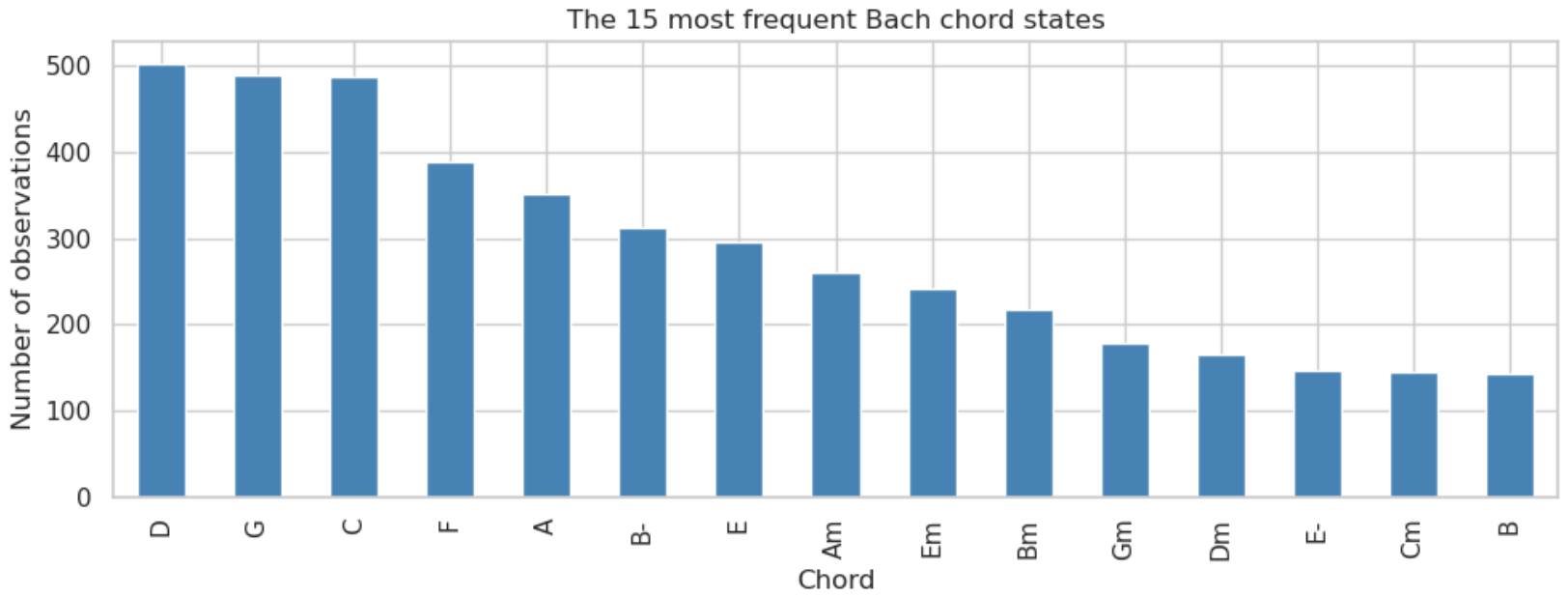
State Space for Bach: 12 chords

```
states = set()
for song in seqs:
    states = states.union(set(song))
states = list(states)

len(states)
# 99
```

- The **state space** is every value the sequence can take
- Here: the 99 distinct chords that appear anywhere in the 62 chorales
- A state is a **modeling choice**. We chose "the chord sounding right now." We could have chosen just the root note, or the key
- The choice is good if the next chord depends only on the current state. We will test that shortly

	song	time	chord
0	000106b_	1	F
1	000106b_	2	C
2	000106b_	3	C
3	000106b_	4	F
4	000106b_	5	F
5	000106b_	6	Dm
6	000106b_	7	Dm
7	000106b_	8	F
8	000106b_	9	F
9	000106b_	10	B-



Coding Exercise: 1,000 Taxis

```
taxi = pd.read_pickle('taxicab.pkl')

len(taxi)           # 1000 cabs
type(taxi[0])       # one pandas Series per cab

taxi[0].tolist()[:10] #example trajectory
```

```
Outside Manhattan, Outside Manhattan, Outside Manhattan,
Outside Manhattan, Outside Manhattan,
Outside Manhattan, Outside Manhattan, Central Park,
Central Park, Upper East Side, ...
```

- Each cab is a sequence of recorded neighborhoods: 22,963,396 observations in all
- The values are the states. The Series index holds neighborhood codes, not times, so keep the values in order and ignore the index
- With your pod:
  - What is the state space? How many states?
  - Build your list of sequences and a list of all states
  - What does one step in the model mean?

2. The Transition Matrix

The Markov Property

- A sequence is a **Markov chain** if the next state depends on the past only through the current state:

$$\text{pr}[X_{t+1} = z' \mid X_t, X_{t-1}, \ldots, X_1] = \text{pr}[X_{t+1} = z' \mid X_t]$$

- Given where you are now, how you got here is irrelevant
- This is what makes **one table** enough. Without it, you would need a rule for every possible history
- It is a strong assumption and often false. Text depends on words far back in a sentence
- We also assume **time-homogeneity**: pr[$X_{t+1} = z' \mid X_t = z$] is the same for every t
- Estimate each conditional probability by counting:

$$\widehat{\text{pr}}[X_{t+1} = z' \mid X_t = z] = \frac{\frac{1}{T}\sum_t \mathbb{I}\{X_t = z, X_{t+1} = z'\}}{\frac{1}{T}\sum_t \mathbb{I}\{X_t = z\}} = \frac{\text{joint}}{\text{marginal}}$$

Bach: One Row of the transition matrix: Where Does D7 Go?

Next chord	Count	$\hat{p}$(next | D7)
G	20	20/58 = 0.34
D7	19	19/58 = 0.33
Gm	11	11/58 = 0.19
E	3	3/58 = 0.05
C	1	1/58 = 0.02
G7	1	1/58 = 0.02
Gm6	1	1/58 = 0.02
A7	1	1/58 = 0.02
E7	1	1/58 = 0.02

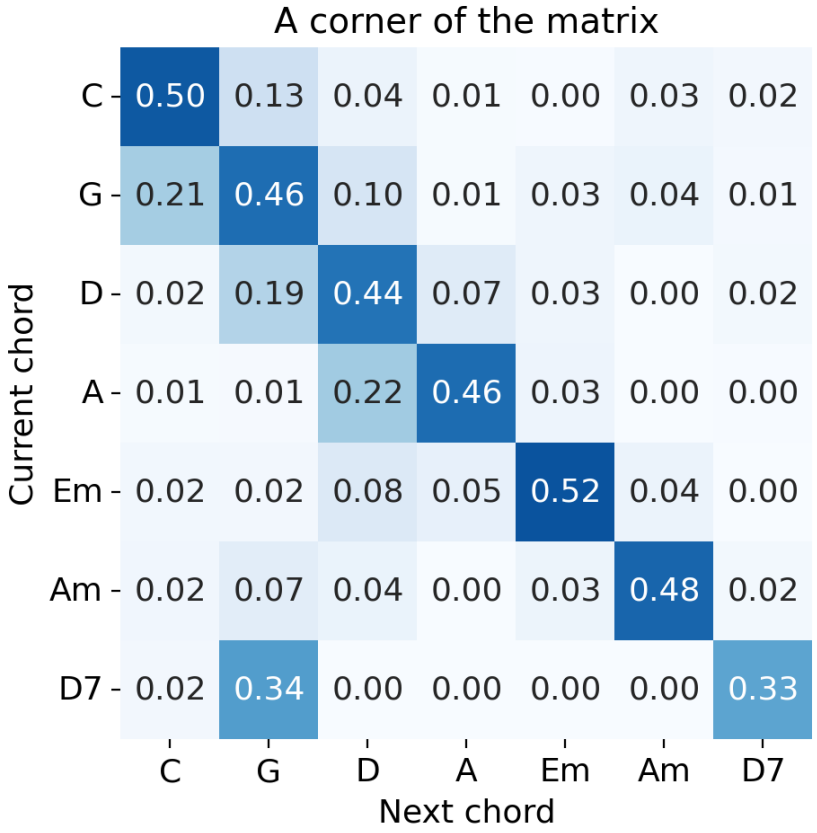
58 moves out of D7. The row sums to 58/58 = 1

- D7 is the chord that "wants" to resolve to G
- In the data: G 34%, stay on D7 33%, G minor 19%
- That one row is **the conditional PMF of the next chord, given the current chord is D7**
- The full matrix is just this, done for every one of the 99 rows

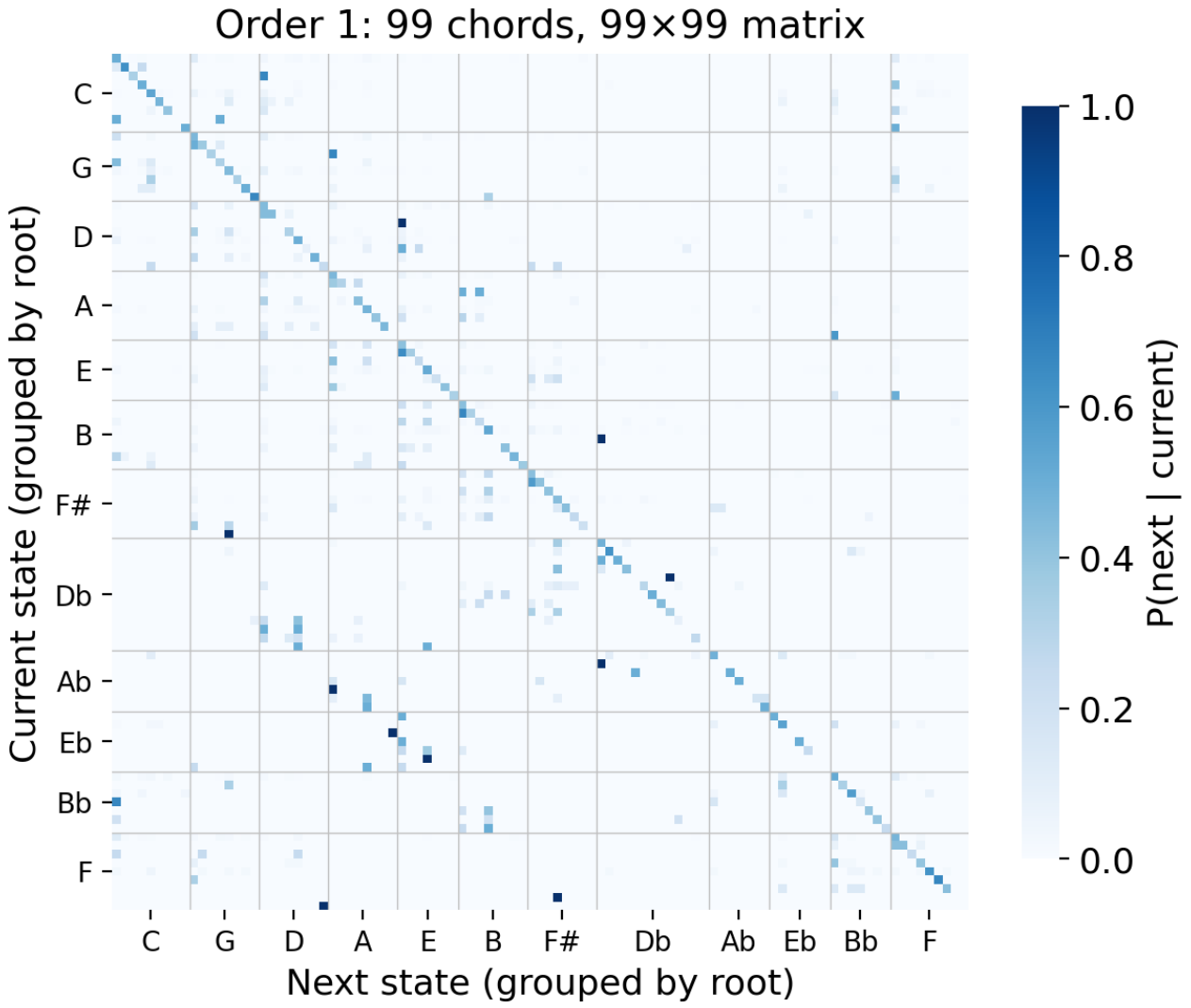


Transition Matrix for Bach:

built with pd.crosstab with normalize= “index”



Left: 7 of the 99 columns, so these rows do not sum to 1 on their own



What does a darker blue square mean in the transition matrix? What does 0 mean?

Let's use the transition matrix to make music!

- Pick a random starting chord
- Using the transition matrix probabilities, randomly jump to a new chord
- Does the result sound like music?

(We will talk about how to do this with code-– simulating a random walk through the Markov chain—on Thursday)

Which is a true Bach chorale? Which is generated by our transition matrix? What is missing from this transition matrix?





Coding exercise: Transition matrix for taxis

3. Markov Chain Order

Order: How Much Memory?

- Remember, to make it Markov, it must be **memoryless**– knowing the current state tells us everything about the next state
  - It is still a **stochastic process**, so the next state is *random*, but having more information about the past doesn't change our probabilities
- If past states *do matter*, we can just add more past days into our "current state" → redefine state by expanding the state space, so the next step *only* depends on the current state

Pair \ Next pair	SS	SR	RS	RR
SS	0.33	0.67	0.00	0.00
SR	0.00	0.00	0.00	1.00
RS	1.00	0.00	0.00	0.00
RR	0.00	0.00	0.50	0.50

8 pair-to-pair moves. Half of each row is 0 by construction

- The **order** of a chain is how many past values make up a state
- Order 2: tomorrow depends on **today and yesterday**, and on nothing earlier once you know those two
- Implement it as an order-1 chain on **pairs**: the state is (yesterday, today)
- Last week's ten days, SSSRRRSSRR, as pairs: SS, SS, SR, RR, RR, RS, SS, SR, RR
- From SR the next pair must start with R: only **RS** or **RR** are possible

Enlarge the State Space: Bach example

- Make the state the last **two** chords:

$$Y_t = (X_{t-1}, X_t)$$

- Now pr[Y_{t+1} | Y_t] uses both chords. **The chain on pairs is order 1 again**
- Same equation, same code. Only the list of states changes
- A move from (a, b) can only go to (b, c). Most cells are 0 by construction

```
pairs = []
for song in seqs:
    pairs.append(
        [song[t-1] + ' ' + song[t]
         for t in range(1, len(song))])

# 'Gm D' -> 'D D' or 'D G' or ...
# never 'Gm D' -> 'A E'
```

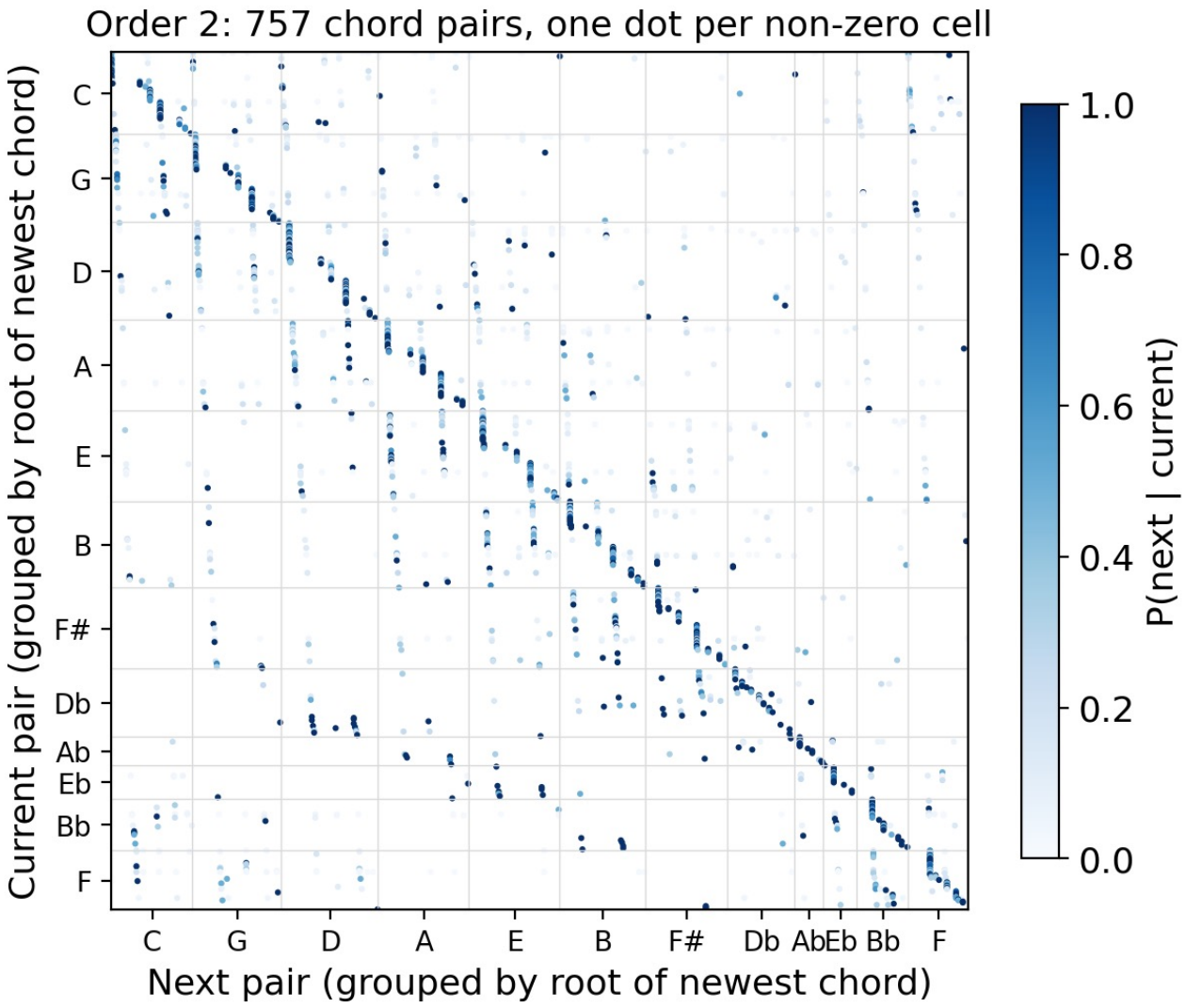
The Cost of Memory

Order k	States	Moves per state	Rows from 1 move	Cells exactly 0
1	99	56.6	7%	92.28%
2	757	7.3	43%	99.69%
3	1,750	3.1	56%	99.91%
4	2,847	1.9	66%	99.95%
5	3,695	1.5	73%	99.97%
6	4,246	1.2	79%	99.97%

From order 2 on, most zeros are structural: from (a, b) the chain can only move to a pair that starts with b, so those cells are 0 whatever the data

- Higher order captures **more real dependence**
- But the 5,665 chords are split across more and more states, so each row is estimated from fewer moves
- At order 6, **79%** of states have only one move to estimate their whole row
- This is the **bias–variance trade-off** again:
  - Order 1, with little memory, may be underfit → high bias
  - But a high order can be overfit to a small number of data points in each state → high variance

The Pair Matrix



- 757 observed pairs, so a 757×757 matrix
- Only 0.31% of its cells are non-zero

Order	States	Moves	Moves / state	Rows from 1 move
1	99	5,603	56.6	7%
2	757	5,541	7.3	43%
3	1,750	5,481	3.1	56%

- **The price of a bigger state**: the same data are spread over more rows, so each row is estimated from fewer moves



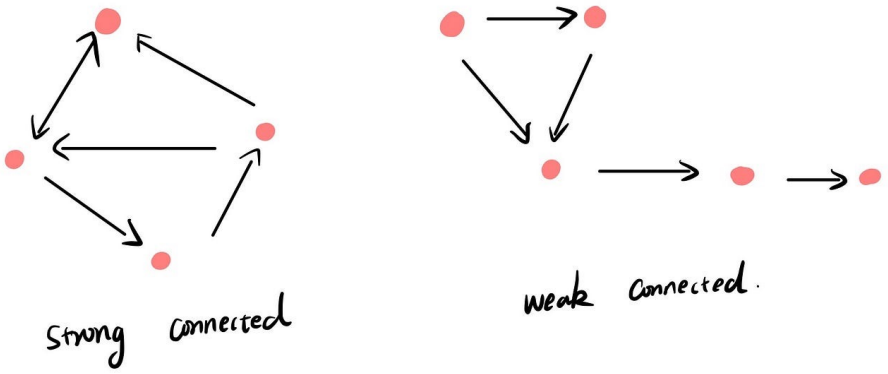
Connected Chains

- A chain is **connected** if every state can reach every other state through moves with non-zero probability
- A chorale in one key never visits most chords: **not connected**. A key signature is a disconnection
- A disconnected chain has groups of states that communicate within, but not across
- Thursday: connectivity is what lets a chain settle into a single long-run distribution

```
import networkx as nx

G = nx.from_pandas_adjacency(
        transition, create_using=nx.DiGraph)
nx.is_strongly_connected(G)
```

Is the order-1 Bach chain connected?
Is the order-2?



Points to Keep in Mind

- **A transition matrix is a conditional PMF**: rows are the current state, and each row sums to 1
- **It rests on two assumptions**: the Markov property and time-homogeneity. Neither is guaranteed by the data
- **Read rows against the marginal**: identical rows would mean the sequence has no memory
- **Order is a trade-off**: more memory means fewer observations per state
- **Unobserved is not impossible**: zero counts need care